

VIO-X 2

9/20/82
#345 AT

COPYRIGHT 1981 - W.W. COMPONENT SUPPLY, INC.

ALL RIGHTS RESERVED

REVISION A1 6/10/81



FULCRUMTM
COMPUTER PRODUCTS

Distributed by:

WW COMPONENT SUPPLY INC. 1771 JUNCTION AVENUE • SAN JOSE, CA 95112 • (408) 295-7171

Introduction

The VIO-X2 video interface board is a powerful addition to an S-100 bus computer. Connected to a standard video monitor and keyboard, it provides the functions of a block mode terminal at a considerably lower price.

The plug-into-the-bus nature of the VIO-X2 provides other advantages in addition to low cost. First, you can pick the monitor you wish to use. If you like a big display, you can attach a giant 25 inch tube. If you don't have much room, a 9 inch (or smaller!) monitor will work just as well. Most any monitor that runs at 15,750 hertz will work with the VIO-X2, which outputs both composite and separate video.

Another advantage of the VIO-X2 over standard terminals is the fact that you can pick the keyboard you want. Many inexpensive terminals have terrible keyboards which quickly wear your fingers to the bone. If you have a favorite keyboard, it doesn't matter what the physical configuration is. As long as it outputs ASCII characters in parallel form, it can work with the VIO-X2. The VIO-X2 will accept positive or negative true data from the keyboard; it works with a positive or negative keyboard strobe, either pulse or level; and it can provide any voltage (positive or negative) to the keyboard when the proper regulators are installed. In addition, most non-ASCII keyboards can be altered to output ASCII, making them usable also.

A third major advantage of the plug-into-the-bus configuration is performance. There is no communication line, no baud rates, no UARTS. So, the only limit on the speed at which the terminal receives characters is how fast it can get them on the screen. This turns out to be very fast indeed: for a normal terminal to keep up with the VIO-X2, it would have to run about 80,000 baud!

Here is a brief rundown on the VIO-X2 specifications:

Interface: IEEE 696 (S-100) Compatible. Host interface is via two I/O ports, status and data; no host system memory space is required. The status and data ports are addressible anywhere in the system, and the status port is configurable to be compatible with most serial interface boards.

Screen Format: 24 rows of 80 characters each; plus a 25th line for status information and system messages.

Up to 16 lines of text may be recalled after they have scrolled off the top of the screen, providing a total of 40 display lines in the terminal at any time.

Character format: 7x7 dot matrix for upper case characters, 7x9 for lower case with descenders, contained in a 10x10 character cell.

Terminal control hardware:

- 8085 microprocessor
- 4K of firmware contained in 2732 ROM
- 4K of RAM for screen memory, etc.
- Intel 8275 CRT controller chip
- 17.496 MHz crystal controlled oscillator

Monitor Interface:

Both separate and composite video.

Both positive and negative vertical sync are provided in separate video mode.

Horizontal frequency is 16,200 Hz.

Vertical frequency is 60.0 Hz.

Video Attributes: Setable on a character-by-character basis, up to 16 fields per line. The following attributes are provided:

- Blinking
- Inverse Video
- Underline or Alternate Character Set
- Dim

...or any combination of the above.

Beeper output: 1.62 KHz output directly drives an 8 ohm speaker. The duration of the beep is .25 seconds.

A light pen interface is provided, allowing the host system to read the row and column of the light pen.

Firmware features:

In addition to the normal dumb terminal features, the VIO-X2 has many functions which give it great power for text editing and data entry. Some of these features are as follows:

a) Three modes of operation.

The VIO-X2 has three distinct modes of operation, each of which serves a specific function.

The first mode is **Conversational** mode, which provides the functions of a standard terminal. Each keyhit is transmitted to the host system as it occurs. Scrolling and rollup/rolldown are conversational mode functions.

The second mode is **Block Unprotect**. In this mode, keyhits are not transmitted, but are instead written directly to the screen. This mode allows the user to create a screenful of data without help from the host system. The host then reads the screen on command from the user. This is the mode to use for block mode text editing.

The third mode of operation is called **Block Protect**. In this mode, protected fields can be created on the screen for data entry purposes. For example, a "form" may be painted on the screen, which the user fills with information; the system, on command from the user (by hitting a function key) then reads the information entered and processes it. Only the data entered by the user is sent to the system; the contents of the form itself are not transmitted, which reduces the system software required.

The form is written as protected characters and therefore cannot be erased inadvertently by the user.

b) Local Editing features

In Block mode, the following on-screen editing functions are available:

- 1) Clear screen
- 2) Clear to end-of-line/screen/field
- 3) Insert/Delete character
- 4) Block Insert mode (existing characters bump over to make room as new characters are typed)

5) Column tab stops: the following functions are provided:

- a) set tab stop
- b) clear tab stop
- c) clear all tab stops
- d) forward tab
- e) back tab

In Protect mode, tab searches for the beginning of the next unprotected field. Similarly, back tab searches for the beginning of the previous unprotected field.

6) User settable margin bell: sounds the audible beeper when a character is entered at the bell column.

In addition, insert line and delete line are available to the system to implement split screen scrolling.

c) Miscellaneous features

The VIO-X2's 25th line performs a number of useful functions:

- 1) The status field displays the current mode of the terminal;
- 2) The clock display is a 24 hour clock which is both settable and readable by the host;
- 3) The remainder of the 25th line can be written by the host, for displaying system status or warnings.

The VIO-X2 offers a transparent mode in which the terminal ignores control characters, displaying them on the screen (using special characters) instead of performing the assigned function (if any). This feature is extremely useful for debugging programs which transmit escape sequences and control characters to the terminal for special effects.

The ability of the VIO-X2 to roll the screen down, recalling lines which have scrolled off the top of the screen, can be most convenient at times, particularly in light of the unusually high speed of this terminal (which can make it difficult to stop the display in time). A related feature is the line feed delay which can be set, limiting the scroll rate to something more reasonable.

VIO-X2 Configuration

The VIO-X2 plugs into any IEEE S-100 bus computer. It contains a keyboard connector, a light pen input, both composite and separate video outputs, and a speaker port.

1) I/O ADDRESS

Before attempting to use the board, its I/O address switches have to be set. They are located on the DIP switch at the bottom of the board. The VIO-X2 uses two contiguous I/O ports; the lower of the two is always an even-numbered port.

Port address bit 7	xxxxxx	Closed = 1
Port address bit 6	xxxxxx	"
Port address bit 5	xxxxxx	"
Port address bit 4	xxxxxx	"
Port address bit 3	xxxxxx	"
Port address bit 2	xxxxxx	"
Port address bit 1	xxxxxx	"
Status/Data swap	xxxxxx	Closed for status = even

To set the address, set the 7 address bits as shown. This addresses the port pair to start at the desired address.

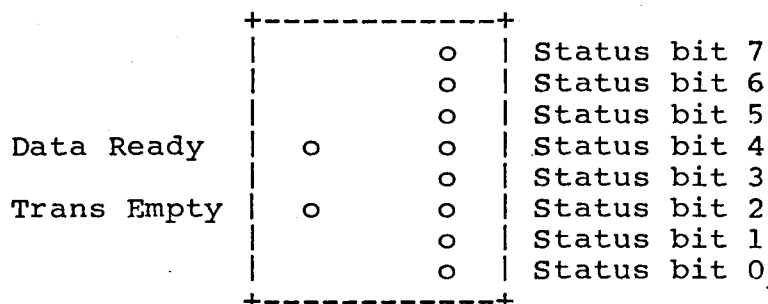
The Status/Data swap switch must also be set: closed for status on even port, data on odd port; open for status on odd port, data on even port.

Here is a typical example: to set the board for IMSAI SIO compatibility, the board is set to I/O ports 2 and 3, with data on port 2 and status on port 3. To do this, the switches are set as follows:

Switch 7 OPEN
Switch 6 OPEN
Switch 5 OPEN
Switch 4 OPEN
Switch 3 OPEN
Switch 2 OPEN
Switch 1 CLOSED
Status/Data swap OPEN

2) STATUS BITS

The bits in the status register must be configured next. The jumper area above and to the right of the Dip switch is used for this purpose. The pads in this area are defined as follows:

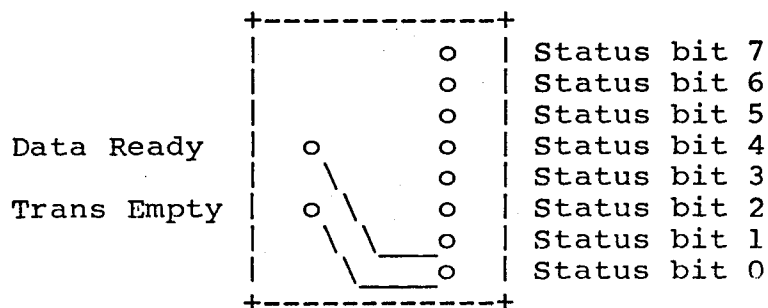


The pad labelled "Data Ready" carries a signal which goes true whenever the terminal has a character in its output buffer, ready to be read by the system.

The "Trans Empty" (transmitter buffer empty) pad goes true when the terminal has read the previous character from its input buffer and the host may write the next character.

To configure the Status port for a particular system, a jumper is connected from each pad on the left to one pad on the right, so that the Data Ready and Transmitter Buffer Empty status bits appear in the proper bits of the status byte.

For example, the SIO compatible configuration must look like an 8251 USART, which has Data Ready on bit 1 of the status port, and Transmitter Buffer Empty on bit 0 of the status port. A jumper is therefore connected from the Trans Empty pad to the bottom pad on the right; another jumper is connected from the Data Ready pad to the next pad up on the right:



3) Video Configuration

The VIO-X2 is capable of driving most standard monitors: either RS-170 type composite or most 15,750 Hz separate video monitors, including Ball Brothers. The horizontal sync frequency is 16,200 Hz; the vertical sync frequency is 60 Hz.

a) Composite Video:

To configure the terminal for composite video, connect the following pins with jumpers:

Jumper area B (near upper DIP switch): connect bottom two pins together.

Jumper area G (near lower Dip switch): only two pins here, connect together.

b) Separate Video:

Jumper Area B: Connect upper pins together

Jumper Area C: For active high vertical sync, connect middle pin to top pin. For active low vertical sync, connect middle pin to bottom pin.

Jumper Area G must not have a jumper when using a separate video monitor.

4) Interrupt Configuration

The last piece of hardware configuration is setting the interrupt jumpers. If your system does not use interrupts, this step is unnecessary.

The interrupt jumper area is at the bottom left of the board, near the big heatsink. The numerous pads in this area have the following functions:

Vectored Interrupt 7	/	o	o	
Vectored Interrupt 6		o	o	RxRDY Unmasked
Vectored Interrupt 5		o	o	
Vectored Interrupt 4		o	o	TxRDY
Vectored Interrupt 3		o	o	
Vectored Interrupt 2		o	o	RxRDY
Vectored Interrupt 1		o	o	
Vectored Interrupt 0		o	o	PINT

The eight Vectored Interrupt lines and the PINT line are connected to the S-100 VI7 through VIO and PINT lines.

The other three pads (RxRDY, TxRDY and RxRDY Unmasked) are 40 ma, open collector, active low signals capable of driving the S-100 interrupt lines.

RxRDY goes low when a character is available and the RxRDY Enable flipflop is on (this is explained later). It goes high as soon as the host reads the character from the output buffer.

TxRDY goes low when the terminal is ready to receive a character, and the TxRDY Enable flipflop is on. It goes high when a character is written to the terminal, and goes low again as soon as the terminal is ready for the next character.

Neither of these interrupt signals ever goes active unless the respective interrupt enable flipflop on set. However, RxRDY Unmasked bypasses the RxRDY Interrupt Enable flipflop, going low whenever a character is available whether or not that flipflop is set.

To configure the interrupts for your system, you must decide which vectored interrupt lines you intend to use; or, you may use the system PINT line directly. If you choose this route, you have the option of tying both RxRDY and TxRDY interrupts to PINT, and reading the status register to determine whether the interrupt was caused by TxRDY or RxRDY; or, RxRDY only can be tied to PINT, allowing incoming characters to interrupt the system, and polling the status port for output. The most flexible method is to use two separate vectored interrupt lines for RxRDY and TxRDY, but this requires a vectored interrupt controller in the system. The choice is yours.

Once you decide on the interrupt scheme, connect jumpers from the interrupt sources (RxRDY, TxRDY, RxRDY Unmasked) to the appropriate bus interrupt lines and you're done.

This completes the configuration as far as the host interface is concerned.

3) OPTIONS

There are several other parameters which also need to be set:

- 1) Scroll Delay: This is a three bit value which determines the default delay time between successive line feeds.
- 2) Cursor Type: Selects between a blinking or non-blinking cursor.
- 3) Keyboard Data Invert: allows a keyboard with inverted data to be used.
- 4) Keyboard Strobe Invert: allows a keyboard with either positive or negative strobe to be used.
- 5) Underline enable/Alternate Character set enable: allows either the Underline or the Alternate Character set feature to be used.

Switch Assignments:

Underline Enable	xxxxxx	Closed = Enabled
Keyboard Strobe Invert	xxxxxx	Closed = Rising Edge Latch
-spare-	xxxxxx	
Cursor Blink/Nonblink	xxxxxx	Closed = Blink
Keyboard Data Invert	xxxxxx	Closed = Non-Inverted
Scroll Speed<2>	xxxxxx	See Table
Scroll Speed<1>	xxxxxx	See Table
Scroll Speed<0>	xxxxxx	See Table

Scroll Speed<2:0> cause the terminal to wait a certain amount of time before performing a scroll. This time delay does not affect any cursor manipulation which does not result in a scroll, such as cursor positioning, carriage return, or line feed when the cursor is not at the bottom of the screen.

The delay time is read from the switches at reset time, but can be changed via software, using the ESC D sequence.

The switches set the delay time according to this table:

<2>	<1>	<0>	Resulting Delay (ms)
C	C	C	0.0
C	C	O	33.3
C	O	C	66.7
C	O	O	100.0
O	C	C	133.3
O	C	O	166.7
O	O	C	200.0
O	O	O	233.3

The keyboard strobe invert switch is not needed for keyboards whose strobe is a pulse which is shorter than the duration of the data valid time. Either switch setting will work equally well (assuming the strobe is really a pulse, not a level which is active whenever a key is down). For other keyboards, set the switch as follows:

If the keyboard data is valid at the low-to-high transition of the strobe, set the invert switch to ON.

If the keyboard data is valid at the high-to-low transition of the strobe, set the invert switch to OFF.

For keyboards whose strobe is a level which lasts more than 100 milliseconds or so (including those which stay on until the key is released), set the invert switch as follows:

If the strobe is active high, set the invert switch to ON.

If the strobe is active low, set the invert switch to OFF.

The Cursor Blink/Nonblink switch is read at reset time and sets the blinkingness of the cursor. If the switch is closed, the cursor blinks. The switch is only read at reset time, so changing the setting of the switch won't do anything to the cursor until a reset is done, either by cycling power or hitting system reset.

The Keyboard Data invert switch is provided for keyboards which generate negative true data. When this switch is open, the keyboard data is inverted by the terminal firmware. See the appendix for more on keyboards.

The Underline Enable switch should normally be set to ON. This enables the underline video attribute to function. When the alternate character set option is installed, the switch must be set to OFF, since the alternate character set is activated by the same attribute which had activated the underline; so if underline is left enabled, all alternate characters will also be underlined.

VIO-X2 Hardware Interface

The VIO-X2 plugs into the S-100 bus and acts as an I/O device. Because of the I/O mapped nature of the VIO-X2, no memory space is taken away from the system and a full 64K of memory can be used. The interface to the system is the same as that of a UART, making the VIO-X2 compatible with 99% of all S-100 systems currently using a UART/serial terminal for I/O.

The terminal communicates with the host system through two I/O ports which occupy contiguous I/O addresses in the S-100 system.

One port is the "status" port: the system reads this port to establish handshaking with the terminal, and writes to the port to set the interrupt mask bits (if required).

The other port is the data port: the system writes to this port to send characters to the screen, and reads the port to get characters typed on the keyboard (there are other times when the system reads the data port, but that doesn't matter at the moment).

Status Read:

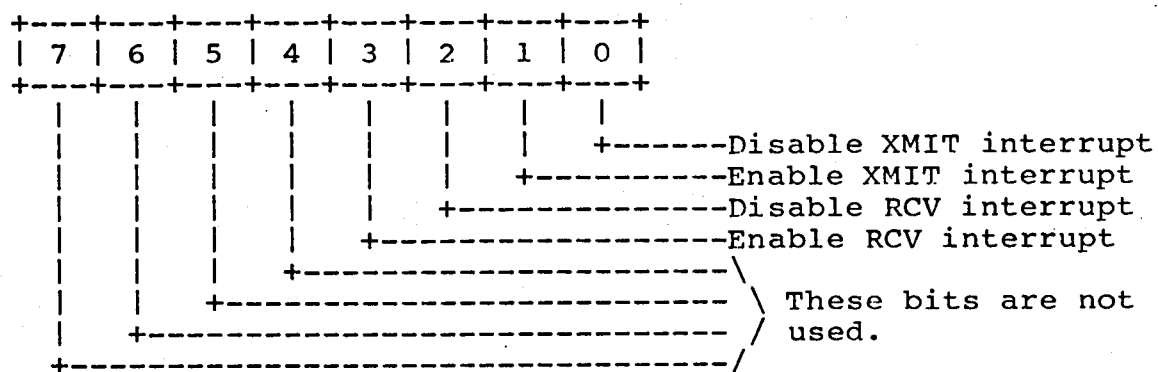
In order to determine if the terminal is ready to receive a character from the host, or if it has a keyboard (or other) character available to the host, the status port provides two status bits called RxRDY (also called "Data Ready") and TxRDY (also called "Transmitter Buffer Empty"). These bits may appear in any position in the status byte, determined by jumpers which are set at configuration time. None of the other bits in the status port are used, and are read as ones.

The TxRDY bit is high whenever the terminal is ready to receive a character. When the host writes a character to the terminal, TxRDY will go low and stay low until the terminal processor has read the character out of its input register. TxRDY will then go high and a new character can be written.

The RxRDY bit goes high whenever the terminal has a character to transmit to the host, for whatever reason (could be a keyboard character, a response to a read cursor command, etc). Data read from the terminal data port is valid only when the RxRDY bit is on. The act of reading the data automatically resets the RxRDY bit.

Status Write:

In order to set up the VIO-X for interrupt operation (under MP/M for example), it is necessary to enable and disable transmit and receive interrupts in the terminal. This is done by writing to the status port. The bits in the byte written to the status port set the interrupt enable flipflops as follows:



If both the enable and disable bits for one of the interrupt enable flipflops are on during a status write, then that flipflop will toggle to the opposite state.

VIO-X2 Control Characters

There are 32 control characters in the ASCII character set. The terminal responds to some (but not all) of them. The characters which the terminal notices are:

Char	Hex	Meaning	Action Taken
@	00H	Null	Ignored by terminal
D	04H	Roll Down	Scrolls screen down one line
G	07H	Bell	Beeeps Beeper
H	08H	Backspace	Bumps cursor to the left
I	09H	Tab	Skip to tab or unprotected field
J	0AH	Line Feed	Bumps cursor down one line
K	0BH	Cursor Up	Bumps cursor up one line
L	0CH	Cursor Right	Bumps cursor to the right
M	0DH	Car Return	Returns cursor to left side
U	15H	Roll Up	Scrolls screen up one line
Z	1AH	Clear Screen	Clears screen to blanks
[	1BH	Escape	Initiates an escape sequence
]	1DH	Back Tab	Skips to previous tab/unprot field
^	1EH	Home	Homes cursor
_	1FH	New Line	Carriage return/line feed

While the terminal does not respond to any other control characters, you can generate any of them by holding down CTRL and hitting the appropriate key. The terminal will then transmit that control character to the host.

Explanation of control characters:

a) CTRL-@ (NULL)

The terminal does not respond to null characters. However, some programs transmit nulls after requesting time-consuming operations (such as clear-screen) in order to provide a time delay which allows the terminal to finish up before the next real character arrives. This technique is often necessary with asynchronous terminals, but the VIO-X2 needs no software delays because it does not have to keep up with a comm line. Because of the full handshaking interface with the host, new characters will not be transmitted until the terminal is ready for them. So, the host remains synchronized with the terminal, even during the slowest operations.

b) CTRL-D (Roll Down) and CTRL-U (Roll Up)

These functions make use of the 40 line display feature. Only 24 lines can be displayed at a time, so some way to look at the hidden lines is needed. This is accomplished with Roll-Up and Roll-Down.

The screen is like a "window" of 24 lines which can be moved up and down the 40 line screen. When the window moves up, the screen appears to roll down, and vice versa.

When the screen is cleared, this 24 line window is positioned at the top of the 40 line display. As the screen fills with characters, the window stays at the top until the cursor reaches the bottom of the screen and a scroll occurs. The window then moves down one line, scrolling the screen up one line.

This process continues until all 40 lines are full. The window is now at the bottom of the 40 line display. Since the window cannot move down any more, the top line is cleared to blanks (you cannot see this line since the window is at the bottom) and the entire 40 line display is rolled up. This blanked line then appears at the bottom of the screen. As long as the window is at the bottom, scrolling is done this way.

This means that up to 40 lines of text can be saved in the terminal memory before the oldest lines start to disappear. The host can move the window up (CTRL-D) or down (CTRL-U) to display lines which have rolled off the top of the screen. Thus CTRL-D scrolls the screen down, while CTRL-U scrolls it up.

Note that the window cannot be moved past the boundaries of the 40 lines, so rollup/rolldown stops when the boundary is reached.

c) CTRL-G (Bell)

When the terminal receives CTRL-G, it beeps its audible beeper.

d) CTRL-H (Backspace)

CTRL-H bumps the cursor one space to the left. If the cursor is already at the left side of the screen, it wraps around to the right side, moving up one line. If it is in the top left corner of the screen, it does not move.

e) CTRL-I (Tab)

CTRL-I performs the same function as ESC J. See the Escape Sequence section for a detailed description of tabs.

f) CTRL-J (Line Feed)

CTRL-J moves the cursor down one line if it is not already at the bottom of the screen. If it is at the bottom, it stays there; but the screen scrolls up one line, leaving a blank line at the bottom.

g) CTRL-K (Cursor Up)

CTRL-K moves the cursor up one line if it is not already at the top of the screen. If it is at the top, it stays there; and if the window is not at the top of the 40 line display, the screen rolls down one line. (If the window is at the top, CTRL-K does nothing).

h) CTRL-L (Cursor Right)

CTRL-L moves the cursor one space to the right, wrapping and scrolling as needed. It differs from <space> in that it does not write a blank to the screen, but only moves the cursor.

i) CTRL-M (Carriage Return)

CTRL-M moves the cursor to the left side of the current line (current unprotected field in Protect Mode).

j) CTRL-Z (Clear Unprot to Spaces)

CTRL-Z performs exactly the same function as "ESC ;". See the Escape Sequence section for a detailed explanation of Clear Unprot to Spaces.

k) CTRL-[(Escape)

Escape initiates an escape sequence. See the following section.

l) CTRL-] (Back Tab)

CTRL-] performs exactly the same function as ESC I; refer to the Escape Sequence section for a detailed description of Back Tab.

m) CTRL-^ (Home Cursor)

CTRL-^ sets the cursor to the home position on the screen, or, in Protect Mode, to the first unprotected position on the screen.

n) CTRL-_ (New Line)

CTRL-_ has exactly the same effect as the sequence CR, LF. The cursor is moved to the beginning of the line following the current line.

VIO-X2 ESCAPE SEQUENCES

An escape sequence consists of at least two characters, the first being the ESC character (1BH). The second character determines which escape sequence is to be executed. Some escape sequences need more characters as part of their processing, but there is always a minimum of two.

Here is a summary of all VIO-X2 escape sequences. The meanings are explained in detail below.

Char	Meaning	Explanation
"	Keyboard Lock	Disables all keyhits until Unlock
#	Keyboard Unlock	Re-enables keyboard
&	Protect mode On	Turns on protect mode
'	Protect mode Off	Turns off protect mode
(	Write protect Off	Turns off write prot char mode
)	Write protect On	Turns on write prot char mode
*	Clear all to nulls	Clears entire screen to nulls
+	Clear all to spaces	Clears entire screen to spaces
,	Turn on 25th line	Enables 25th line status field
.	Turn off 25th line	Disables 25th line status field
4	Send line (unprot)	Transmits line (foreground only)
5	Send page (unprot)	Transmits page (foreground only)
6	Send line (all)	Transmits line (all characters)
7	Send page (all)	Transmits page (all characters)
8	Write to 25th line	Writes chars to 25th line
9	Set 25th line att	Sets video attribute of 25th line
:	Clr to nulls (unprot)	Clears unprot chars to null
;	Clr to space (unprot)	Clears unprot chars to space
<	Read time	Transmits contents of clock
=	Load Cursor	Positions cursor on screen
>	Set time	Sets clock
?	Read cursor	Transmits cursor position
B	Set block mode	Turns on block mode
C	Set conv. mode	Turns on conversational mode
D	Set scroll delay	Sets delay between scrolls
E	Insert line	Inserts blank line at cursor
G	Set Margin Bell	Sets bell column = cursor column
H	Cursor Off	Turns off visible cursor
I	Back tab	Tab backward from cursor position
J	Tab	Tab forward from cursor position
K	Cursor on	Turns disabled cursor back on
L	Read light pen	Transmits light pen position
M	Clear tab	Clears column tab stop at cursor
Q	Insert character	Inserts space at cursor position
R	Delete line	Deletes line at cursor position
S	Clear all tabs	Clears all column tab stops
T	Erase line	Erases to end-of-line

U	Set transparent mode	Turns on display of control chars
V	Set tab	Sets column tab stop
W	Delete character	Deletes character under cursor
Y	Erase page	Erases to end-of-screen
Z	Set video attribute	Sets active attribute
DEL	Read Firmware Version	Returns firmware version to host

a) Lock keyboard (ESC #) and Unlock keyboard (ESC ")

Keyboard lock disables the keyboard so that no keyhits are transmitted in conversational mode, and nothing is written to the screen in block mode. This feature is generally used in applications which read data from the screen, to prevent keyhits from transmitting spurious characters to the application program.

When the keyboard is locked, the message KBD LOCK appears in bright inverse video characters in the 25th line status field. In addition, the cursor is shut off.

Should the keyboard become locked accidentally, it can be unlocked locally by typing CTRL-@ (null).

Keyboard unlock has the opposite effect. The keyboard is reactivated, the KBD LOCK message is replaced with the appropriate status message, and the cursor re-appears.

b) Protect mode On (ESC &) and Off (ESC ')

Protect mode is used to prevent certain fields of characters on the screen from being erased or over-written. These fields are called Protected fields, and always appear dim (no matter what video attributes are in effect). Protected fields are written to the screen by setting Write Protected Character mode (see below).

When Protect mode is on, the cursor is prevented from ever sitting on a dim character. If the cursor hits a protected field when moving forward or down, it skips forward until it encounters an unprotected field. Similarly, if the cursor encounters a protected field while moving to the left or up, it skips backward until it finds an unprotected character to sit on.

Thus, it is not possible to type over a protected character in Protect mode. (In unprotect mode, there is no difference between dim and bright characters as far as protection is concerned).

Protect mode also affects some other operations, including:

- Erase line/page
- Insert/Delete line
- Insert/Delete character
- Clear unprot to null/space
- Send line/page
- Tab/Back tab

How these operations are affected is detailed in the individual explanation for each function.

Protect mode can only be set while Block mode is on. In Conversational mode, the ESC & sequence is ignored.

In Protect mode, it is not possible to generate video attributes. Any characters written to the screen in Protect mode will take on the attribute of the field in which they are written. In particular, protected characters cannot be written while Protect mode is on, since that implies writing dim characters.

Whenever Protect mode is on, the character 'p' in the lower right-hand corner of the screen will be upper case. When protect mode is off, the 'p' is lower case to indicate unprotect mode.

Protect mode allows a "form" to be created on the screen. The form will contain various protected and unprotected fields. Generally, each unprotected field (or "Data Entry" field) corresponds to a protected field which indicates what information is to be entered into the unprotected field.

For example, a form can be created which interacts with a mailing list program. A form of this type would contain data entry fields for name, address, phone number, and any other information of interest. Each data entry field would have a protected field before it to prompt the user to type the correct information into it.

The "Name" field, for example, could have the word "Name" in dim (protected) characters, followed by a data entry field of about 25 unprotected spaces, enough room to type a person's name. (The data entry field could be underlined or inverse video to make it more visible). The user would position the cursor at the beginning of the data entry field (generally by using the Tab key) and enter the name. Any other fields on the screen are filled the same way.

Note that no interaction with the host system software is required to do this. The terminal is in Block mode, and all keyboard processing is local. The only host system task thus far has been to paint the form on the screen. This greatly simplifies the software, which would otherwise have to keep track of the cursor position and which areas on the screen are protected.

When all fields are complete, the operator hits a "function key". This is a special key which causes the terminal to transmit a "function key sequence" to the host computer. The VIO-X2 can support up to 64 such function keys, each of which generates a different function key sequence and can have a different meaning to the application program.

When the function key is hit, the host receives the function key sequence and acts accordingly. A key labeled "SAVE" would, for example, cause the host to read the screen contents (using one of the Read Unprot functions), format the data properly, and write it to the disk file. The following record (if any) could then be read and displayed on the same form (the form need only be written once).

c) Write protected character mode On [ESC)] or Off [ESC (]

All characters written to the screen during Write Prot Char mode will appear dim. These dim characters become protected characters when Protect mode is invoked (see above).

Protect mode is only available when Block mode is on. However, dim characters may be written in either Conversational or Block mode. This allows dim characters to be written to the screen, which effectively provides an extra video attribute for programs which do not use Block mode.

When Write Prot Char mode is on, the character "w" at the bottom right corner of the screen will be upper case. When this mode is turned off, the "w" is lower case.

d) Clear all to Nulls (ESC *) and Clear all to Spaces (ESC +)

These two functions are identical to each other except for the fill character which is written to the screen.

When either functions is invoked, the entire 40-line screen is cleared to the appropriate fill character, and the cursor is placed at the home position. Also, the 24-line window is moved to the top 24 lines.

Both Protect mode and Write Prot Char mode are reset to Off after a Clear All command.

e) Clear Unprot to Nulls (ESC :) or Spaces (ESC ;))

Again, these two functions are identical to each other, except for the fill character which is written to the screen.

When Protect mode is off, the effect is similar to Clear All, except that only 24 lines are cleared (to blanks or nulls). The lines cleared are those at the top of the current 40 line screen. The remaining 16 lines are unaffected. The cursor is placed at the home position.

When Protect mode is on, the 24 line window is not moved. All unprotected fields on the screen are cleared to the appropriate fill character, and the cursor is moved to the first unprotected character on the screen. Further, no video attributes anywhere on the screen are affected; for example, an underlined unprotected field will remain underlined after the Clear Unprot; and an unprotected field with one or more video attributes imbedded in it will retain those attributes.

f) Turn 25th line On (ESC ,) and Off (ESC .)

These two functions control the status field at the right side of the 25th line. The "ESC ." sequence makes the clock (if on) and terminal status message invisible. Note that if the clock was running, it is still running and can still be read by the host.

The "ESC ," sequence re-activates the status field, restoring it to the state it was in prior to the shutoff command.

g) Send Line Unprot (ESC 4) and Send Line All (ESC 5)

These two commands provide a means to read the screen data. While they are functional in both Conversational and Block mode, their main use is in conjunction with Block mode.

The basic function of Send Line is to transmit to the host the characters on the current (cursor) line, starting at the beginning of the line and stopping at the character under the cursor.

If protect mode is off, Send Line Unprot and Send Line All perform exactly the same function, which is to transmit the current line from the beginning to the cursor location (inclusive). A carriage return character is sent to indicate completion. Video attributes (including dim) encountered in the line are ignored.

When the sequence is complete, the cursor is placed at the beginning of the next line. Note that this is not the case in Protect mode.

If Protect Mode is on, Send Line Unprot and Send Line All no longer perform the same function. Send Line Unprot transmits only the unprotected characters in the line, while Send Line All transmits all characters (ending at the cursor of course).

With Protect Mode on, Send Line Unprot works this way: a scan is started at the left side of the current line. Unprotected characters are transmitted, as they are encountered, to the host. If a protected field is found, an ASCII FS (field separator, hex 1C) is transmitted and the scan skips over the protected characters. (If the first character in the line is protected, an FS character is transmitted before anything else). When another unprotected field is encountered, character transmission starts again. This process continues until the character under the cursor has been transmitted; a carriage return character is then sent to indicate completion, and the process is done. The cursor does not move (different from non-protect mode -- see above).

With Protect Mode on, Send Line All works this way: a scan is started at the left side of the current line. Characters are sent as they are encountered. Whenever a transition from unprotected characters to protected characters happens, the

terminal inserts "ESC)" into the data stream. Similarly, whenever a transition from protected to unprotected characters happens, the sequence "ESC (" is inserted. Note that if the first character in the line is protected, "ESC)" is sent before anything else.

Note that nulls are never transmitted by Send Line, whether or not the terminal is in Protect mode. Spaces, however, are transmitted normally.

h) Send Page Unprot (ESC 5) and Send Page All (ESC 7)

The Send Page functions are similar to the Send Line functions. However, instead of the current line only being sent, all characters from the beginning of the screen to the cursor position are transmitted to the host.

When Protect mode is off, these two commands perform exactly the same function. Beginning at the home position, characters are transmitted to the host. Each time the scan jumps from the end of one line to the beginning of the next line, a new-line character (CTRL-^, hex 1F) is inserted into the data stream. This continues until the character under the cursor has been transmitted. A carriage return is then transmitted to indicate completion.

Video attributes (including dim) cause no special action in unprotect mode and are ignored.

When Protect mode is on, the two functions are no longer equivalent. Send Page Unprot transmits only unprotected characters, while Send Page All transmits all characters, protected or not.

Send Page Unprot works like this in Protect mode: the scan starts at the home position; if this position is protected, an ASCII Field Separator character (hex 1C) is transmitted first. Transmission begins with the first unprotected character on the screen. Each time a transition from unprotected to protected characters is detected, another FS character is sent and the scan skips to the next unprotected character. This process continues until the character under the cursor has been sent. A carriage return character is then sent to indicate completion.

Video attributes do not cause any special action and are simply skipped over during this process.

Send Page All works this way in Protect mode: the scan starts at the home position. The first thing transmitted is an escape sequence which indicates protected or unprotected field. For unprotected, the sequence ESC (is transmitted; for protected, ESC) is transmitted. Next, the video attribute for this field

is sent. The format is ESC Z <att>; <att> is a character which defines the attribute of the field. See the Set Video Attribute description for the meaning of the <att> character.

Once these two sequences have been sent, screen characters are sent until a new field is encountered, either because of a new video attribute, or because of a change in the protect status (or both). If due to a video attribute, the ESC Z <att> sequence is sent and transmission continues. If due to protect/unprotect, the ESC (or ESC) sequence is sent to indicate the protect status of the new field; then the ESC Z <att> sequence is sent to indicate the video attribute of the new field (whether or not the new attribute is the same as before). This process continues until the character under the cursor has been sent. A carriage return character is then transmitted to indicate completion.

Note that nulls are never transmitted by Send Page, whether the terminal is in Protect mode or not. Spaces, however, are transmitted normally.

i) Write to 25th line (ESC 8)

After the terminal receives ESC 8, all subsequent characters are written to the 25th line until either a control character is received, or 57 characters have been received. During the 25th line write, the cursor stays where it was prior to the ESC 8. If the write sequence is terminated by a control character, that control character is not otherwise acted upon. After termination of the sequence, terminal operation returns to normal.

It is possible to write the 25th line with a video attribute. All characters written will have the same attribute, which is set by the ESC 9 (Set 25th line att) sequence. The setting of the 25th line attribute must be done before the actual write to the 25th line.

j) Set 25th line attribute (ESC 9)

This sequence sets the video attribute in which subsequent writes to the 25th line will occur. The complete sequence is ESC 9 <att>, where <att> is a character which determines which attribute is to be generated. For the attributes which correspond to different values of <att>, please refer to the description of Set Video Attribute.

k) Set Time (ESC >) and Read Time (ESC <)

These two functions allow the 25th line clock to be set and read.

Setting the clock is done by transmitting ESC > followed by six ASCII characters which will become the setting of the clock. For example, to set the clock to 12:34:56, the following sequence is sent:

ESC > 1 2 3 4 5 6

Note that the clock is a military time clock only, so there is no concept of a.m. and p.m.

If any of the six characters following the ESC > is "@", the clock is reset; that is, if the clock display was already present on the 25th line, it is cleared. Furthermore, the "@" aborts the set-time sequence, returning the terminal to normal operation even if six characters have not yet been sent.

The terminal does not check for illegal (non-digit) characters when setting the clock. Nothing will stop you from putting letters and punctuation marks in the display (you may want to try it just for fun). It is the responsibility of the host system software to do error checking in this case.

To read the time, ESC < is sent to the terminal, which responds by transmitting the six ASCII digits which represent the current clock time to the host, most significant digit first. A carriage return is then sent to indicate completion. Note: if the clock is not running when the read-time command is issued, six blanks are transmitted, followed by carriage return.

l) Read cursor (ESC ?) and Load cursor (ESC =)

Read Cursor causes the terminal to transmit to the host two ASCII characters which represent the screen row and column on which the cursor is currently sitting. A carriage return is then sent to indicate completion. (The values of the row and column characters are described in the table below).

Load cursor allows the cursor to be placed anywhere on the screen. The complete sequence is ESC = y x where y is the row and x is the column in which the cursor is to be placed.

The following table translates the ASCII value of y and x into absolute row and column numbers:

space	0	4	20	H	40	\	60
!	1	5	21	I	41	]	61
"	2	6	22	J	42	^	62
#	3	7	23	K	43	~	63
\$	4	8	24	L	44		64
%	5	9	25	M	45	a	65
&	6	:	26	N	46	b	66
'	7	;	27	O	47	c	67
(	8	<	28	P	48	d	68
)	9	=	29	Q	49	e	69
*	10	>	30	R	50	f	70
+	11	?	31	S	51	g	71
,	12	@	32	T	52	h	72
-	13	A	33	U	53	i	73
.	14	B	34	V	54	j	74
/	15	C	35	W	55	k	75
0	16	D	36	X	56	l	76
1	17	E	37	Y	57	m	77
2	18	F	38	Z	58	n	78
3	19	G	39	[	59	o	79

m) Set Block mode (ESC B) and Set Conversational mode (ESC C)

In Conversational mode, each key hit is transmitted to the host as it occurs. Nothing is written to the screen unless the host software echos the character back to the terminal.

In Block mode, typing on the keyboard is not detected by the host, since the keyboard is in local mode. The screen is filled with the appropriate information without help from the host software; then a special function key is hit. This causes the terminal to transmit a function key sequence to the host, which responds by reading the screen. Thus, information is transmitted in blocks, instead of on a character-by-character basis.

Local keyboard operation is not the only difference between Conversational and Block modes. The other differences are as follows:

- 1) In Conversational mode, the screen scrolls when a line feed occurs with the cursor at the bottom of the screen. In Block mode, the screen does not scroll; instead, the cursor wraps back around to the top of the screen.
- 2) Protect mode can only be set when Block mode is already on.
- 3) The roll-up and roll-down functions are only available in Conversational mode.

4) The margin bell is only active in Block mode.

5) The terminal responds to Tab and Back Tab commands in Block mode only.

Note that setting Block mode automatically clears all column Tab stops.

n) Set Scroll Delay (ESC D)

For applications where the scroll rate is too fast to be easily readable, this sequence will provide a delay which the terminal will wait before it executes a scroll. The complete sequence is ESC D d, where d is a character in the range SPACE to "?". This character tells the terminal how many 16.7 ms intervals (i.e., frames) to wait before it will execute a scroll. The character assignment is as follows:

Char	Frames	Max LF/sec	Char	Frames	Max LF/sec
SPACE	0	Very, very many	0	16	3.8
!	1	60.0	1	17	3.5
"	2	30.0	2	18	3.3
#	3	20.0	3	19	3.2
\$	4	15.0	4	20	3.0
%	5	12.0	5	21	2.9
&	6	10.0	6	22	2.7
'	7	8.6	7	23	2.6
(	8	7.5	8	24	2.5
)	9	6.7	9	25	2.4
*	10	6.0	:	26	2.3
+	11	5.5	;	27	2.2
,	12	5.0	<	28	2.1
-	13	4.6	=	29	2.1
.	14	4.3	>	30	2.0
/	15	4.0	?	31	1.9

Note that this delay only affects actual scrolling of the screen. It does not affect anything else; in particular, cursor positioning and line feeds which do not result in a scroll. Roll up and roll down are also unaffected.

o) Insert Line (ESC E) and Delete Line (ESC R)

Insert Line moves all lines, from the cursor line to the bottom of the screen, down one line. The bottom line is lost, and the cursor line is cleared to spaces. The cursor does not move.

Delete line moves all lines, from the cursor line to the bottom of the screen, up one line. The cursor line is lost, and the bottom line is cleared to spaces. Again, the cursor does not move.

If there are lines below the bottom of the screen (the result of a roll-down command), they are unaffected by Insert Line and Delete Line.

Both Insert Line and Delete Line are disabled in Protect Mode.

p) Set Margin Bell (ESC G)

The margin bell feature allows the beeper to signal that a certain column on the screen has been written to. For example, if the bell column is set to column 70, then the beeper will beep when the cursor is on column 70 and a character is entered.

Note that the margin bell is only functional in Block mode, and that the beeper only sounds when a character is entered from the keyboard (not when characters come in from the host).

To set the margin bell column, the cursor is placed in the desired column (the row is unimportant), and the sequence ESC G is sent to the terminal. The bell column is now set to the current cursor column. (This can also be done locally from the keyboard -- see appendix).

If the cursor is in column 0 (leftmost column) when the Set Margin Bell command is received, the margin bell is disabled. This is the setting of the margin bell column on reset or power-up.

q) Cursor On (ESC K) and Cursor Off (ESC H)

Cursor Off disables the video representation of the cursor. Logically, however, the cursor still exists and is fully functional. Cursor On re-enables the cursor, unless the keyboard is locked (keyboard lock also turns off the cursor).

r) Tab (ESC J) and Back Tab (ESC I)

The Tab and Back Tab functions work differently in Protect and Unprotect modes. The two different modes are explained below.

In Unprotect mode, Tab and Back Tab respond to columnar tab stops which have been previously set by the user. Tab jumps to the next tab stop on the right, while Back Tab jumps to the next tab stop on the left.

If there are no tab stops set, Tab leaves the cursor at the beginning of the next line, while Back Tab leaves the cursor at the beginning of the current line.

In Protect mode, columnar tab stops are no longer active. Tab and Back Tab now search for unprotected fields, as follows:

Tab starts a scan at the cursor position, searching for a protected field. This search continues until a protected field is found, or the end of the screen is reached, whichever occurs first. If the end of the screen is reached, the cursor jumps to the first unprotected character on the screen, and the process is finished. If, however, a protected field is found before the end of the screen, then the search continues until an unprotected field is found (whether or not the end of the screen is reached). When the unprotected field is found, the cursor is placed at the beginning of it.

Back Tab works like this: if the cursor is at the beginning of the unprotected field in which it is located, then the cursor skips backward to the beginning of the previous unprotected field, or to the bottom right corner of the screen (whichever occurs first). If, however, the cursor is not at the beginning of its unprotected field, it jumps to the beginning of the current field.

s) Read Light Pen (ESC L)

When the terminal receives the Read Light Pen command, it waits for input from the light pen. Once this occurs, the row and column of the light pen are transmitted to the host as two characters "yx", where y is the row and x is the column of the light pen (see the explanation of Read cursor and Load cursor for the meanings of y and x). A carriage return is sent to indicate completion.

If a character is sent to the terminal while it is waiting for the light pen, the light pen command is cancelled and the character is processed normally.

The terminal does not flash the screen when the light pen is activated, which means that the pen must be placed over an area of the screen which is already emitting light. Also, delays in the light pen circuitry and the monitor phosphor persistence can cause significant errors which must be corrected in software. Beware of slow phosphor monitors.

CAUTION

t) Set Tab (ESC V), Clear Tab (ESC M), Clear all Tabs (ESC S)

These functions are only available in Block Unprotect mode.

Set Tab sets a tab stop at the current cursor column. From this point on, the cursor can be placed in this column in response to a Tab or Back Tab command.

Clear Tab clears the tab stop at the current cursor position. If there is no tab stop set, the Clear Tab does nothing.

Clear all Tabs clears all tab stops.

These functions are all available as local keyboard functions as well. Refer to the appendix for details.

u) Insert Character (ESC Q) and Delete Character (ESC W)

In Unprotect mode, Insert Character inserts a space at the cursor position. The cursor character, as well as all characters to the right on the current line, move one character to the right. The last character on the line is lost.

Delete line is similar, except that the cursor character is deleted and all characters to the right move one space to the left to fill the gap. A space appears at the rightmost position on the line.

If there are no protected fields on the line, then these two functions work the same in both Protect and Unprotect mode. If, however, there is a protected field to the right of the cursor, then the insertion/deletion terminates at the protected field; that is, the last character in the current unprotected field is lost when inserting, and a space appears in the last character position of the current unprotected field when deleting.

Both of these functions are available locally from the keyboard. See the appendix for details.

v) Erase Line (ESC T) and Erase Page (ESC Y)

In Unprotect mode, Erase Line clears the current line to blanks, starting at the cursor position and ending at the end of the line. The cursor does not move.

Similarly, Erase Page clears the screen to spaces, starting at the cursor and ending at the end of the screen. The cursor stays put.

In Protect mode, Erase Line clears the current line to spaces, starting at the cursor position and ending at the end of the current unprotected field, or the end of the line, whichever occurs first. Again, the cursor does not move.

In Protect mode, Erase Page clears all unprotected fields, starting at the cursor position, and ending at the end of the screen. No protected fields are affected; furthermore, any video attributes inside unprotected fields are unaffected. The cursor does not move.

w) Set Transparent Mode (ESC U)

Transparent mode is useful for debugging programs which interact with the terminal in a non-trivial fashion. No control character is acted upon; instead, they are displayed on the screen using special characters. The only control character not displayed is NULL, which appears as a blank.

When the terminal is in transparent mode, the normal status display on the 25th line is replaced by the word "XPARENT" in bright inverse video characters.

To exit transparent mode, the sequence ESC ESC must be sent to the terminal. The terminal then returns to normal mode.

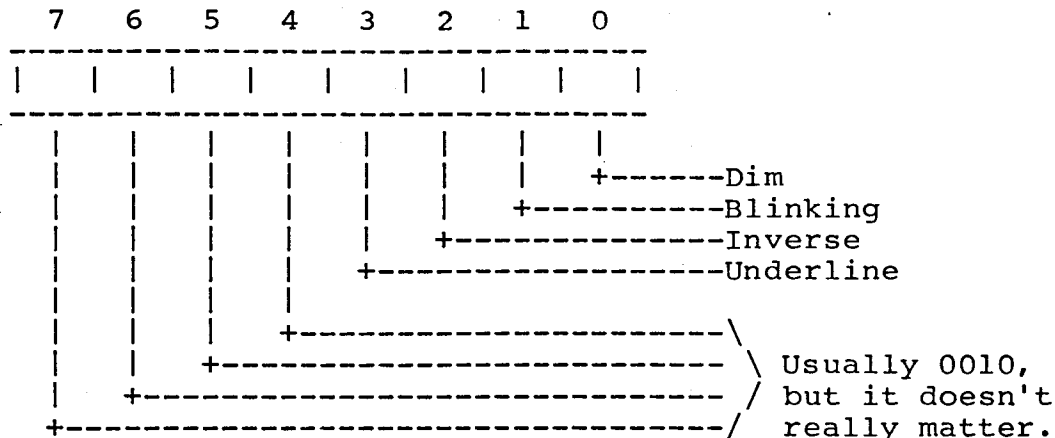
Note: Transparent mode can be set and reset locally from the keyboard. See the keyboard appendix for details.

x) Set Video Attribute (ESC Z)

Video attributes in the VIO-X2 are set on a character-by-character basis. This means that characters written to the screen will have the current attribute, regardless of the attributes of the characters nearby (there is a limit to this, as explained below). The current attribute is set by transmitting the sequence ESC Z <att>, where <att> is a character which determines what the new attribute will be:

<att>	Video Attribute
space	Normal
!	Dim
"	Blinking
#	Dim, Blinking
\$	Inverse
%	Dim, Inverse
&	Blinking, Inverse
'	Dim, Blinking, Inverse
(	Underline
)	Dim, Underline
*	Blinking, Underline
+	Dim, Blinking, Underline
,	Inverse, Underline
-	Dim, Inverse, Underline
.	Blinking, Inverse, Underline
/	Dim, Blinking, Inverse, Underline

Another way to look at this is to break the <att> character into individual bits as follows:



A "one" in any attribute bit means that attribute is activated.

The number of video attributes that can exist on one line is not unlimited. This is because the video hardware does not really generate attributes character-by-character, but rather on a field by field basis. The hardware is limited to 16 fields per line.

Assume there is a line filled with normal video characters. If the current attribute is set to, say, inverse, and a character is written somewhere in the middle of this line, then that character will be inverse, surrounded by normal characters. There are therefore three fields on the line (a field of normal characters, followed by a field of one inverse character, followed by another field of normal characters).

So, by writing one character, the number of fields on the line has jumped from one to three. Applications which use video attributes must be careful not to exceed 16 fields. Note that there is always at least one field in the line, so only 15 changes are allowable.

The terminal does not check for too many fields, so the maximum number can be exceeded. This is not serious, however, because a correct program will not try to write more than 16 fields anyway; the result would be wrong, whether or not the terminal checked for overflow. The malfunctions which result from field overflow are not fatal, and are corrected by clearing the screen.

y) Read Version (ESC DEL)

When the terminal receives ESC DEL (7FH), it returns a two character sequence which represents the version number of the firmware ROM currently installed in the terminal. The first character is a letter A-Z and indicates the firmware type. The second character is a digit 0-9 and indicates the revision level

of the firmware. A carriage return character is sent to indicate completion.

For the VIO-X2, the initial version number is "D0". If a firmware change is required, it will become "D1", etc.

INTERRUPT OPERATION

The VIO-X2 contains two flipflops caled TxINT ENABLE and RxINT ENABLE. When the TxINT ENABLE flipflop is on, the terminal will assert an interrupt whenever it is ready to receive a character. Similarly, when RxINT ENABLE is on, the terminal will interrupt whenever it has a character to transmit to the host.

These flipflops (particularly TxINT ENABLE) are necessary for proper interrupt operation, since it must be possible to disable interrupts sometimes. For example, if the TxDY interrupt were not disable-able, then a costant interrupt would be asserted whenever characters were not being sent to the terminal.

At power-on and after a reset, the RxINT ENABLE flipflop is set ON, while the TxDY ENABLE flipflop is set to OFF. The software can change the setings of these flipflops by writing to the status port:

- Bits 7-4 are unused.
- Bit 3 = Enable RxINT
- Bit 2 = Disable RxINT
- Bit 1 = Enable TxINT
- Bit 0 = Disable TxINT

Note that the settings of these flags are not readable by the software.

The RxINT ENABLE flipflop is normally left in the enabled state, since, in the idle state, the terminal has nothing to send and the interrupt will not be asserted.

The TxINT ENABLE flipflop is normally left in the disabled srtate, since, in the idle state, the terminal is ready to receive a character and the interrupt would therefore be asserted all the time.

That is why the two flipflops are set the way they are after a reset.

If your system has a vectored interrupt controller with a mask register for disabling individual interrupts, then you may leave TxINT and RxINT enabled all the time, using the mask register on the interrupt controller to disable unwanted TxINT interrupts.

Here is an example of interrupt driven I/O. It is not simple like polling I/O, but does allow terminal I/O to occur cleanly while other processes are taking place. The following assumptions apply:

- The Receive interupt routine resides at 0008H;
- The Transmit interrrupt routine resides at 0010H;
- The terminal is configured as an IMSAI SIO.

```

;
; Receiver Interrupt sends us here, jump to where we have more
; room.
;
RST1:      ORG 0008H
           JMP RXINT

;
; Transmitter Interrupt sends us here, jump to more roomy place
;
RST2:      ORG 0010H
           JMP TXINT
           .
           .
           .

;
; Receiver Interrupt Routine
;
RXINT:     PUSH PSW                ; SAVE ALL CLOBBERED REGISTERS
           PUSH B                  ;
           PUSH H                  ;
           LHLD INPOINT           ; GET INPUT POINTER
           IN DATA               ; INPUT DATA FROM TERMINAL
           MOV B,A                ; SAVE FOR NOW
           LDA TERMCHR            ; GET TERMINATION CHARACTER
           CMP B                  ; COMPARE WITH CHAR JUST READ
           JZ FULL                ; IF IT MATCHES, TERMINATE READ
           MOV M,B                ; DOESN'T MATCH, WRITE TO BUFFER
           INX H                  ; INCREMENT BUFFER POINTER
           SHLD INPOINT           ; SAVE FOR NEXT TIME
           LDA CNTREAD            ; GET CHARACTER COUNT
           INR A                  ; INCREMENT IT
           STA CNTREAD            ; STORE AGAIN
           LDA READCNT            ; GET INPUT COUNTER
           DCR A                  ; DECREMENT IT
           STA READCNT            ; STORE AGAIN
           JZ FULL                ;
RXDONE:    POP H                  ; RESTORE CLOBBERED REGISTERS
           POP B                  ;
           POP PSW                ;
           EI                     ; DON'T FORGET!!
           RET                    ; DONE.
FULL:     MVI A,RXOFF             ; GET HERE WHEN DONE WITH READ
           OUT STATUS             ; TURN OFF RXINT (OPTIONAL)
           MVI A,0FFH            ;
           JMP RXDONE             ; REALLY DONE.

```

```

;
; Transmitter Interrupt Routine
;
TXINT:    PUSH PSW                ; SAVE ALL CLOBBED REGISTERS
          PUSH H                  ;
          LDA  OUTCNT             ; GET OUTPUT COUNT
          ORA  A                  ; CHECK IF EQUAL TO ZERO
          JZ   EMPTY             ; IF SO, DO NOT TRANSMIT
          DCR  A                  ; NOT ZERO, DECREMENT OUT COUNT
          STA  OUTCNT             ; STORE AWAY FOR NEXT TIME
          LHLD OUTPNT             ; GET OUTPUT BUFFER POINTER
          MOV  A,M                ; LOAD CHARACTER FROM BUFFER
          OUT  DATA              ; OUTPUT TO TERMINAL
          INX  H                  ; INCREMENT BUFFER POINTER
          SHLD OUTPNT             ; STORE AWAY FOR NEXT TIME
TXDONE:   POP  H                  ; RESTORE CLOBBED REGISTERS
          POP  PSW                ;
          EI                      ; RE-ENABLE INTERRUPTS
          RET                     ; DONE.
EMPTY:    MVI  A, TXOFF           ; NO MORE CHARS IN BUFFER
          OUT  STATUS             ; TURN OFF TXINT ENABLE FLAG
          MVI  A, OFFH            ; SET WRITE COMPLETED FLAG
          STA  TXCMPLT           ;
          JMP  TXDONE             ; REALLY DONE.

```

The RXINT routine is invoked whenever a character is ready from the terminal (assuming RxINT ENABLE is set). The characters are read and fill a memory buffer (pointed to by INPOINT). The number of characters read is contained in CNTREAD.

If a character is read which matches the termination character TERMCHAR, the read is terminated and the RXCMPLT flag is set to let the software know. A typical termination character would be <cr>. When the read terminates, the RxINT ENABLE flipflop is reset to prevent any more characters from coming in, until the next buffer is readied. In addition, the flag RXCMPLT is set to inform the software that the read is finished.

If a termination character never shows up, the buffer fills until the maximum number of characters (as specified by READCNT at the time the read is initiated) have been read. The read then terminates as above, with CNTREAD containing the number of characters read (equal, in this case, to READCNT when the read was initiated).

The TXINT routine is invoked whenever the terminal is ready to receive a character (assuming the TxINT ENABLE flipflop is set, of course). The routine empties a buffer one character at a time, until the location OUTCNT is exhausted. When this happens, the TxINT ENABLE flipflop is reset and the flag TXCMPLT is set to inform the software that the write has finished.

The following routine sets up an input buffer of 80 characters. The termination character is set to carriage return. For this example, the routine loops waiting for RXCMPLT to go true, although a real application could be doing other things during this time.

```

;
; Buffer Read Routine
;
BUFRD:   LXI   H,BUFFER           ; CHARACTERS WILL SHOW UP HERE
        MVI   A,80               ; READ 80 CHARACTERS MAX
        STA   READCNT           ;
        XRA   A                 ; INIT. COUNT READ TO ZERO
        STA   CNTREAD           ;
        STA   RXCMPLT           ; CLEAR READ COMPLETED FLAG
        MVI   A,0DH             ; CARRIAGE RETURN CHARACTER
        STA   TERMCHAR          ; SET TERMINATION CHARACTER
        MVI   A,RXON            ; ENABLE RXINT ENABLE
        OUT   STATUS            ;
BUFRDL1: LDA   RXCMPLT           ; WAIT FOR READ COMPLETE FLAG
        ORA   A                 ;
        JZ    BUFRDL1           ;
        .
        .                       ; READ COMPLETE, PROCESS BUFFER...
        .                       ; COUNT READ IS IN CNTREAD.

```

The following routine takes a buffer BUFFER of 80 characters and outputs it to the terminal. The number of characters in the buffer is stored in OUTCNT, and the pointer to buffer is OUTPNT. Again, the software loops until the write is finished, although this is not the typical case.

```

;
; Buffer Write Routine
;
BUFWRT:  LXI   H,BUFFER           ; POINT AT OUTPUT BUFFER
        SHLD  OUTPNT             ; STORE INTO OUTPUT POINTER
        MVI   A,80               ; 80 CHARACTERS TO WRITE
        SHLD  OUTCNT            ; STORE INTO OUTPUT COUNTER
        XRA   A                 ;
        STA   TXCMPLT           ; CLEAR TRANSMIT COMPLETE FLAG
        MVI   A,TXON            ;
        OUT   STATUS            ; SET TXINT ENABLER FLAG
BUFWRL1: LDA   TXCMPLT           ; LOOP UNTIL TXCMPLT GOES TRUE
        ORA   A                 ;
        JZ    BUFWRL1           ;
        .
        .                       ; WRITE COMPLETE.
        .

```

Interrupts provide a means of doing I/O while other processes (even other I/O, polled or interrupt driven) are going on. All that is required is to set up the buffer and start the process going. The interrupt routine will take care of the rest. The routines shown here can be improved to handle more complicated functions; for example, the input routine does not handle backspace. You the user should decide upon your personal requirements and create the I/O structure which is the best suited to your system.

Appendix

The keyboard used with the VIO-X2 must be a fully encoded ASCII keyboard. Either positive true or negative true data will work, provided the switch on the terminal pc board is set appropriately (see Configuration section). Functions such as N-key rollover, auto repeat, etc. must be handled by the keyboard itself.

The terminal expects 8 data bits from the keyboard. For normal operation, the most significant bit (bit 7) must be "false", that is, TTL low for positive true data, and TTL high for negative true data.

The terminal firmware supports many local functions which cause no transmission of data to the host, such as clear-screen, roll up/down, etc. These functions are invoked when bit 7 is true. There are 16 different local functions available. The 16 functions and their appropriate codes are as follows:

Key	Key Code	Function
0	B0	Clear unprotect to Spaces
1	B1	Roll down
2	B2	Roll up
3	B3	Enable/Disable 25th line
4	B4	Set/Clear transparent mode
5	B5	Clear all tab stops
6	B6	Insert character
7	B7	Delete character
8	B8	Set bell column
9	B9	Back Tab
:	BA	Turn on Insert Mode
;	BB	Erase to end of line
<	BC	Erase to end of screen
=	BD	Set tab stop
>	BE	Clear tab stop
?	BF	Clear all to spaces

The character under "key" is the character which will generate the appropriate action when its upper bit is set. Thus, any standard keyboard can generate all of the local functions as long as the upper bit can be turned on (generally by an extra control key -- for example, "FLAG I" on the IMSAI IKB-1). For example, to set a tab stop, you hold down the extra control key and type "=". This generates the code OBDH which is the Set Tab Stop code.

function keys

When the FLAG I key is held down, the keys "@" through "~" (codes C0 through FE) generate a function key sequence. The function key is a three character sequence which is transmitted to the host as follows:

First character is SOH (ctrl-A, 01H).

Second character is the character which was typed (40H for "@", 7EH for "~", etc.)

Third character is carriage return.

Note that although the keycode has the upper bit set, the keycode transmitted to the host always has the upper bit off.

Function keys provide the way to get the attention of the computer in block mode, since no other keyhits are transmitted to the host. Each function key is unique because of the second character in the sequence; so, each can be assigned a different function. A total of 63 different function keys can exist (it would be 64, but the code 0FFH does not generate a function key sequence).

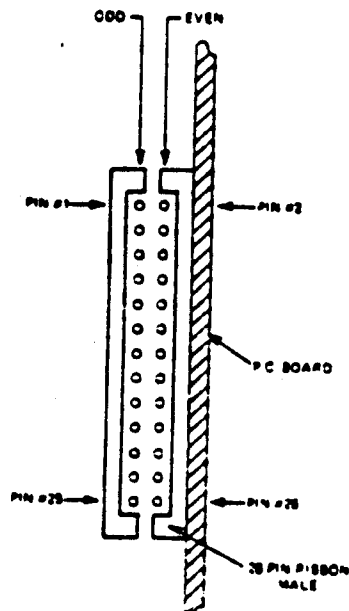
One typical example which is used in the real world is a block mode text editor. The screen is edited locally in block mode, without help from the host system. When the editing is done, a variety of functions are available via function keys, such as roll up, roll down, insert/delete line, next page, previous page, etc. The host reads the function key sequence and performs the action required, and then returns control to the user.

It is always preferable to have a keyboard with a row of function keys along the top to generate the function key sequences, rather than trying to remember that "FLAG I, A" means "Next Page". An overlay can be placed over the function keys to indicate the function of each one.

VIO-X VIDEO TERMINAL
Connections

VIO-X
KEYBOARD INPUT CONNECTOR (J1)

J1 PIN#	DB-25S PIN#	SIGNAL NAME	DESCRIPTION
18	22	DI7	DATA BIT 7
16	21	DI6	DATA BIT 6
14	20	DI5	DATA BIT 5
12	19	DI4	DATA BIT 4
10	18	DI3	DATA BIT 3
08	17	DI2	DATA BIT 2
06	16	DI1	DATA BIT 1
04	15	DIO	DATA BIT 0
22	24	RDY	DATA READY (STROBE)
20	23	RST	RESET
01	01	GND	GROUND
23	12	+5	+5VDC
02	14	+12	+12VDC (OPT)
21	11	SPK	SPEAKER OUT
24	25	-12	-12VDC (OPT)

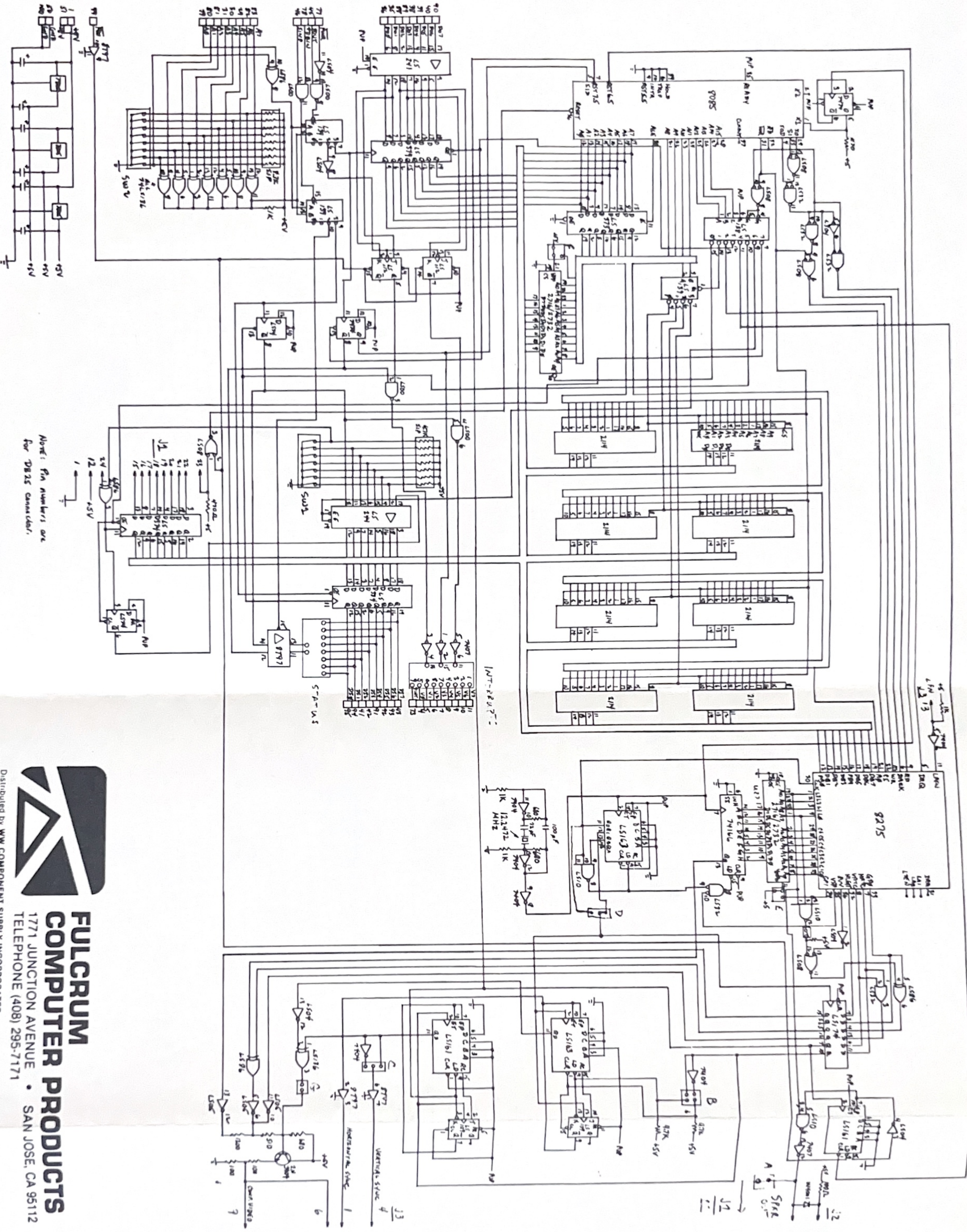


VIO-X
INTERRUPT STRAPPING LIST

PIN#	SIGNAL	DESCRIPTION
01	V7	VECTORED INTERRUPT 7
02	V6	VECTORED INTERRUPT 6
03	V5	VECTORED INTERRUPT 5
04	V4	VECTORED INTERRUPT 4
05	V3	VECTORED INTERRUPT 3
06	V2	VECTORED INTERRUPT 2
07	V1	VECTORED INTERRUPT 1
08	V0	VECTORED INTERRUPT 0
09	INT	INTERRUPT
11	DAV	
13	TBE	
15	ORDY	

VIO-X
CRT VIDEO CONNECTOR (J3)

PIN #	SIGNAL	DESCRIPTION
01	HOR	HORIZONTAL SYNC
02	GND	
03	GND	
04	VER	VERTICAL SYNC
05	GND	
06	VID	VIDEO OUT
07	GND	
08	GND	
09	CVID	COMPOSITE VIDEO
10	N/C	
11	N/C	
12	GND	
13	LPN	LIGHT PEN INPUT



Note: Pin numbers are for DB25 connector.



**FULCRUM
COMPUTER PRODUCTS**

1771 JUNCTION AVENUE • SAN JOSE, CA 95112
TELEPHONE (408) 295-7171

Distributed by WW COMPONENT SUPPLY INCORPORATED